

UNIVERSIDAD AUTÓNOMA DE SANTO DOMINGO

CURSO-UASD (SANTO DOMINGO)

PRESENTACIÓN

Docente(s)

Radhames Silverio y Marlenis Concepción

Materia

Laboratorio de Lenguaje de Programación III (INF 5170)

Tema

Practica 10: Spring Framework
(Describir las funcionalidades de este Framework)

Sección

01

Nombre	Matrícula
Miguel Enmanuel Castillo Garcia	DA1734

Fecha

17/10/2026

Introducción

El desarrollo de aplicaciones empresariales requiere organizar el código, administrar las dependencias entre componentes, conectarse con bases de datos, atender solicitudes web y proteger la información. Cuando estas tareas se implementan desde cero, pueden aumentar la complejidad, el tiempo de desarrollo y el esfuerzo necesario para mantener el sistema. Los frameworks ofrecen estructuras y herramientas reutilizables para resolver problemas frecuentes.

Spring Framework es una plataforma de código abierto ampliamente utilizada para desarrollar aplicaciones con Java. Su diseño modular permite incorporar las capacidades que necesita cada proyecto, desde la administración de objetos y transacciones hasta la construcción de aplicaciones web. En su ecosistema existen proyectos complementarios como Spring Boot, Spring Data y Spring Security, que simplifican tareas habituales del desarrollo moderno.

En este trabajo se describen las principales funcionalidades de Spring Framework, sus características y ventajas, los módulos y proyectos más importantes, ejemplos de aplicación y una reflexión sobre su importancia en el desarrollo de software.

1. ¿Qué es Spring Framework?

Spring Framework es un framework de código abierto para Java que proporciona infraestructura y un modelo de programación para construir aplicaciones, especialmente aplicaciones empresariales. Ofrece mecanismos reutilizables para administrar objetos, configurar componentes, manejar transacciones, desarrollar aplicaciones web e integrar tecnologías de persistencia.

Uno de sus conceptos fundamentales es la inversión de control (IoC). En lugar de que cada clase cree manualmente todos los objetos que necesita, el contenedor de Spring puede crear y administrar los componentes de la aplicación. La inyección de dependencias (DI) es una forma de aplicar este principio: las dependencias se suministran a un componente, por ejemplo, mediante su constructor, en lugar de que este las cree directamente.

Este enfoque reduce el acoplamiento entre clases y facilita las pruebas, el mantenimiento y el reemplazo de implementaciones. Es importante distinguir Spring Framework de proyectos relacionados: Spring Framework constituye la base, mientras que Spring Boot, Spring Data y Spring Security aportan capacidades adicionales para necesidades específicas.

2. Principales características y ventajas

2.1. Características principales

- **Inversión de control e inyección de dependencias:** el contenedor administra los objetos y las relaciones entre ellos.
- **Arquitectura modular:** permite seleccionar las funcionalidades necesarias sin utilizar obligatoriamente todos los componentes.
- **Programación orientada a aspectos (AOP):** ayuda a separar tareas transversales, como auditoría, registro y determinados controles de transacciones.
- **Gestión de transacciones:** facilita coordinar operaciones que deben completarse como una unidad, especialmente en el acceso a datos.
- **Desarrollo web:** Spring MVC permite crear aplicaciones web siguiendo el patrón Modelo-Vista-Controlador y construir endpoints HTTP.
- **Integración:** ofrece soporte para trabajar con bases de datos, bibliotecas de persistencia, mensajería, correo y otras tecnologías.
- **Soporte para pruebas:** proporciona utilidades para probar componentes y aplicaciones, además de integrarse con herramientas del ecosistema Java.
- **Código abierto:** dispone de documentación oficial, guías, ejemplos y una comunidad de desarrolladores.

2.2. Ventajas

- **Mantenibilidad:** la separación de responsabilidades facilita modificar y ampliar funcionalidades.
- **Menor acoplamiento:** los componentes pueden depender de interfaces o contratos, lo que facilita cambiar implementaciones.
- **Productividad:** aporta soluciones reutilizables para tareas técnicas comunes y reduce la repetición de código de infraestructura.
- **Facilidad de pruebas:** los componentes desacoplados pueden probarse de forma aislada y sus dependencias pueden sustituirse por objetos de prueba.
- **Flexibilidad:** su modularidad permite adaptarlo a diferentes requisitos y entornos.
- **Capacidad de crecimiento:** puede emplearse en aplicaciones pequeñas, APIs y sistemas empresariales de mayor tamaño; la escalabilidad final también depende de la arquitectura y la infraestructura.

Estas ventajas no significan que Spring resuelva automáticamente todos los problemas. El equipo debe diseñar una arquitectura coherente, seleccionar y actualizar dependencias, proteger la configuración y realizar pruebas. El framework facilita el trabajo, pero no sustituye el criterio técnico.

3. Explicación de los módulos más importantes

3.1. Spring Core

Spring Core contiene conceptos esenciales de la infraestructura de Spring. Junto con el contenedor IoC y el módulo de contexto, permite configurar, crear y administrar objetos conocidos como beans. La configuración puede realizarse mediante anotaciones, clases Java de configuración y otros mecanismos admitidos por Spring.

Ejemplo: en una aplicación de inventario, Spring puede administrar un servicio de productos y suministrarle el repositorio que necesita. El servicio se concentra en las reglas del negocio sin crear directamente la implementación concreta del repositorio.

3.2. Spring MVC

Spring MVC es el framework web de Spring basado en el patrón Modelo-Vista-Controlador. El modelo representa los datos y la lógica relacionada; la vista presenta la información; y el controlador recibe solicitudes y coordina el procesamiento. En una API REST, el controlador puede devolver datos en JSON en lugar de renderizar una página.

Ejemplo: en un punto de venta, un controlador puede recibir una solicitud para consultar productos, utilizar un servicio para ejecutar la operación y devolver los resultados al cliente.

3.3. Spring Boot

Spring Boot simplifica la creación y ejecución de aplicaciones basadas en Spring. Ofrece configuración automática, dependencias iniciales agrupadas (starters), servidores web integrados en escenarios compatibles y funciones útiles para operar aplicaciones. Su objetivo es reducir la configuración repetitiva y permitir que el desarrollador se concentre en la funcionalidad.

Spring Boot no reemplaza a Spring Framework; se apoya en él y facilita su configuración. Por ejemplo, permite poner en marcha una API web con una configuración inicial reducida y controladores definidos por el desarrollador.

3.4. Spring Data

Spring Data es una familia de proyectos que ofrece un modelo de programación consistente para acceder a diferentes tipos de almacenamiento. Incluye soporte para bases de datos relacionales y no relacionales. Sus repositorios permiten declarar operaciones comunes de consulta y persistencia sin escribir manualmente toda la infraestructura de acceso a datos.

Ejemplo: un repositorio de productos puede ofrecer operaciones para buscar por identificador o recuperar los productos disponibles. Según la tecnología seleccionada, se puede utilizar Spring Data JPA para bases de datos relacionales o Spring Data MongoDB para documentos.

3.5. Spring Security

Spring Security proporciona mecanismos de autenticación, autorización y protección frente a ataques comunes en aplicaciones. La autenticación comprueba quién es el usuario y la autorización determina qué recursos u operaciones puede utilizar.

Ejemplo: en un sistema de ventas, un cajero podría registrar ventas y consultar productos, mientras que un administrador tendría permisos adicionales para modificar inventario o gestionar configuraciones. Estas reglas deben comprobarse en el servidor; ocultar un botón en la interfaz no constituye por sí solo una medida de seguridad.

3.6. Cómo se complementan

Los proyectos pueden combinarse para cubrir responsabilidades diferentes. Spring Boot simplifica la configuración general; Spring MVC recibe solicitudes HTTP; Spring Core administra componentes; Spring Data facilita el acceso a la base de datos; y Spring Security controla el acceso. Esta combinación es frecuente, pero cada proyecto debe incorporar solo las piezas que necesita.

4. Ejemplos de uso del framework en aplicaciones reales

Spring puede utilizarse en diferentes tipos de sistemas. Los siguientes son escenarios habituales de aplicación y no afirmaciones sobre la arquitectura interna de una empresa específica.

- **Comercio electrónico:** administración de productos, inventario, pedidos, pagos e integración con otros servicios.
- **Banca y servicios financieros:** servicios de consulta y operaciones que requieren transacciones, controles de acceso, auditoría y pruebas rigurosas.
- **Sistemas de gestión empresarial:** aplicaciones de ventas, facturación, compras e inventario que necesitan separar reglas del negocio, persistencia e interfaz.
- **APIs para aplicaciones móviles y web:** un backend puede ofrecer endpoints REST consumidos por aplicaciones construidas con tecnologías distintas.
- **Procesamiento e integración de datos:** pueden desarrollarse tareas programadas, importaciones, notificaciones e integraciones con otros sistemas.

4.1. Ejemplo aplicado a un sistema de punto de venta

Considérese una aplicación de punto de venta que administra productos, registra ventas y consulta inventario. Una posible organización sería:

1. **Controlador web (Spring MVC):** recibe la solicitud de registrar una venta y devuelve una respuesta HTTP.

2. **Servicio de ventas (Spring Core):** aplica las reglas del negocio, valida datos y coordina la operación.

3. **Repositorio (Spring Data):** consulta y persiste productos, ventas y detalles en la base de datos.
4. **Seguridad (Spring Security):** comprueba que el usuario esté autenticado y autorizado para ejecutar la operación.

5. **Configuración y ejecución (Spring Boot):** simplifica la configuración y el arranque de la aplicación.

Esta separación ayuda a identificar dónde implementar cada responsabilidad. Si cambia la tecnología de almacenamiento, por ejemplo, las modificaciones deberían concentrarse principalmente en la persistencia y su configuración, evitando mezclar esos detalles con las reglas de ventas. El diseño concreto debe adaptarse a los requisitos y probarse antes de utilizarse en producción.

Conclusión

Considero que Spring Framework es importante en el desarrollo de software porque proporciona una base organizada para construir aplicaciones Java mantenibles, modulares y preparadas para evolucionar. Su aporte no consiste únicamente en ofrecer herramientas, sino también en facilitar la separación de responsabilidades y reducir la dependencia directa entre los componentes.

También considero valioso comprender la diferencia entre Spring Framework y proyectos como Spring Boot, Spring Data y Spring Security. Cada uno atiende necesidades distintas y, cuando se combinan de manera adecuada, permiten desarrollar soluciones completas sin construir desde cero toda la infraestructura técnica.

Como estudiante de Informática, estudiar Spring representa una oportunidad para fortalecer mis conocimientos de Java, arquitectura de software, APIs, persistencia y seguridad. Sin embargo, utilizarlo correctamente requiere comprender los fundamentos de programación y diseño, consultar la documentación oficial, mantener las dependencias actualizadas y aplicar buenas prácticas. En conclusión, Spring es una herramienta relevante para el desarrollo empresarial cuando se utiliza con criterio técnico y de acuerdo con las necesidades del proyecto.

Referencias

- VMware, Inc. (s. f.). Spring Framework. https://spring.io/projects/spring-framework

- Spring. (s. f.). Spring Framework Reference Documentation: Overview. https://docs.spring.io/spring-framework/reference/overview.html

- VMware, Inc. (s. f.). Spring Boot. https://spring.io/projects/spring-boot

- VMware, Inc. (s. f.). Spring Data. https://spring.io/projects/spring-data/

- Spring. (s. f.). Spring Security Reference. https://docs.spring.io/spring-security/reference/

- Spring. (s. f.). Core Technologies. Spring Framework Reference Documentation. https://docs.spring.io/spring/reference/core.html